

R Series: Profiling

In this nineteenth article in the R series, we shall learn about profiling R code.



We will use R version 4.2.1 installed on Parabola GNU/Linux-libre (x86-64) for the code snippets in this article.

```
$ R --version
```

```
R version 4.2.1 (2022-06-23) -- "Funny-Looking Kid"
Copyright (C) 2022 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu (64-bit)
```

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under the terms of the
GNU General Public License versions 2 or 3.
For more information about these matters see
<https://www.gnu.org/licenses/>.

system.time

Consider the 'Bank Marketing Data Set' for a Portuguese banking institution available from the UCI Machine Learning Repository available at <https://archive.ics.uci.edu/ml/datasets/Bank+Marketing>. The data can be used for public research use. There are four data sets available, and we will use the `read.csv()` function to import the data from a 'full-bank.csv' file. The `system.time()` function on an R expression provides the user, system and total time for the program, as shown below:

```
> system.time(bank <- read.csv(file="bank-full.csv", sep=";"))
  user  system elapsed 
0.129   0.000   0.135
```

```
> bank[1:3,]
  age job marital education default balance housing
1  58 management married tertiary      no   2143      yes
no unknown      5
2  44 technician single secondary      no    29      yes
no unknown      5
3  33 entrepreneur married secondary      no     2      yes
yes unknown      5
 month duration campaign pdays previous poutcome y
1  may      261         1    -1         0 unknown no
2  may      151         1    -1         0 unknown no
3  may       76         1    -1         0 unknown no
```

Another example is computing the cross-product of two matrices. The `rmnorm()` function generates values for the normal distribution. It accepts 'n' number of observations along with values for mean and standard deviation. In the following code snippet, we are generating two matrices, 'm' and 'n', with 30 rows and 15,000 columns, and computing the cross-product between them using the `crossprod()` function. This is actually faster than calling 't(m) %*% n'.

```
> m = matrix(rnorm(30*15000, mean=0, sd=4), 30, 15000)
> n = matrix(rnorm(30*15000, mean=0, sd=4), 30, 15000)

> system.time(crossprod(m, n))
  user  system elapsed 
3.716   0.339   4.058
```